EL4001 Assignment Report #01

2D Transformation in Homogeneous Coordinates

Jongwon Kim

September 14, 2026

1 Introduction

This assignment studies 2D geometric transformations using homogeneous coordinates. The provided Python code applies transformations to an image using 3×3 transformation matrices. The assignment also considers the difference between affine transformations and general projective transformations.

The input image used in the experiments is `mona.jpg`. Its width and height are 800 and 1192 pixels, respectively. The main implementation is based on the provided `transform.py` code.

A 2D point (x, y) is represented in homogeneous coordinates as

$$\tilde{\mathbf{p}} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{1}$$

This representation allows translation, rotation, and projective transformations to be expressed using matrix multiplication.

2 Q1. Meshgrid and Homogeneous Coordinates

2.1 Why is `meshgrid` used?

The following line is used in the provided `apply_transformation` function:

```
x, y = np.meshgrid(np.arange(cols), np.arange(rows))
```

The purpose of `meshgrid` is to generate the (x, y) coordinate of every pixel in the image.

For an image with width W and height H,

$$x = 0, 1, \ldots, W - 1 \tag{2}$$

represents the column coordinates, while

$$y = 0, 1, \ldots, H - 1 \tag{3}$$

represents the row coordinates.

Therefore, `meshgrid` creates two 2D arrays with the same spatial size as the image. For example, for pixel location (r, c) in the image,

$$x[r, c] = c, \qquad y[r, c] = r. \tag{4}$$

Here, the image indexing convention is important. NumPy accesses an image using `image[row, column]`, while the geometric coordinate is written as $(x, y) = (\text{column}, \text{row})$.

The two coordinate arrays are then combined with an array of ones:

```
ones = np.ones_like(x)
homogeneous_coords = np.stack([x, y, ones], axis=-1).reshape(-1, 3)
```

This produces homogeneous coordinates of the form

$$\tilde{\mathbf{p}} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \tag{5}$$

for every pixel.

For the input image used in this assignment,

$$H = 1192, \qquad W = 800, \tag{6}$$

so the total number of pixels is

$$1192 \times 800 = 953600. \tag{7}$$

Consequently, after the reshape operation, `homogeneous_coords` has the shape

$$(953600, 3). \tag{8}$$

Thus, `meshgrid` does not itself perform the transformation. Instead, it efficiently generates the coordinates of all image pixels so that the same transformation matrix can be applied to them at once.

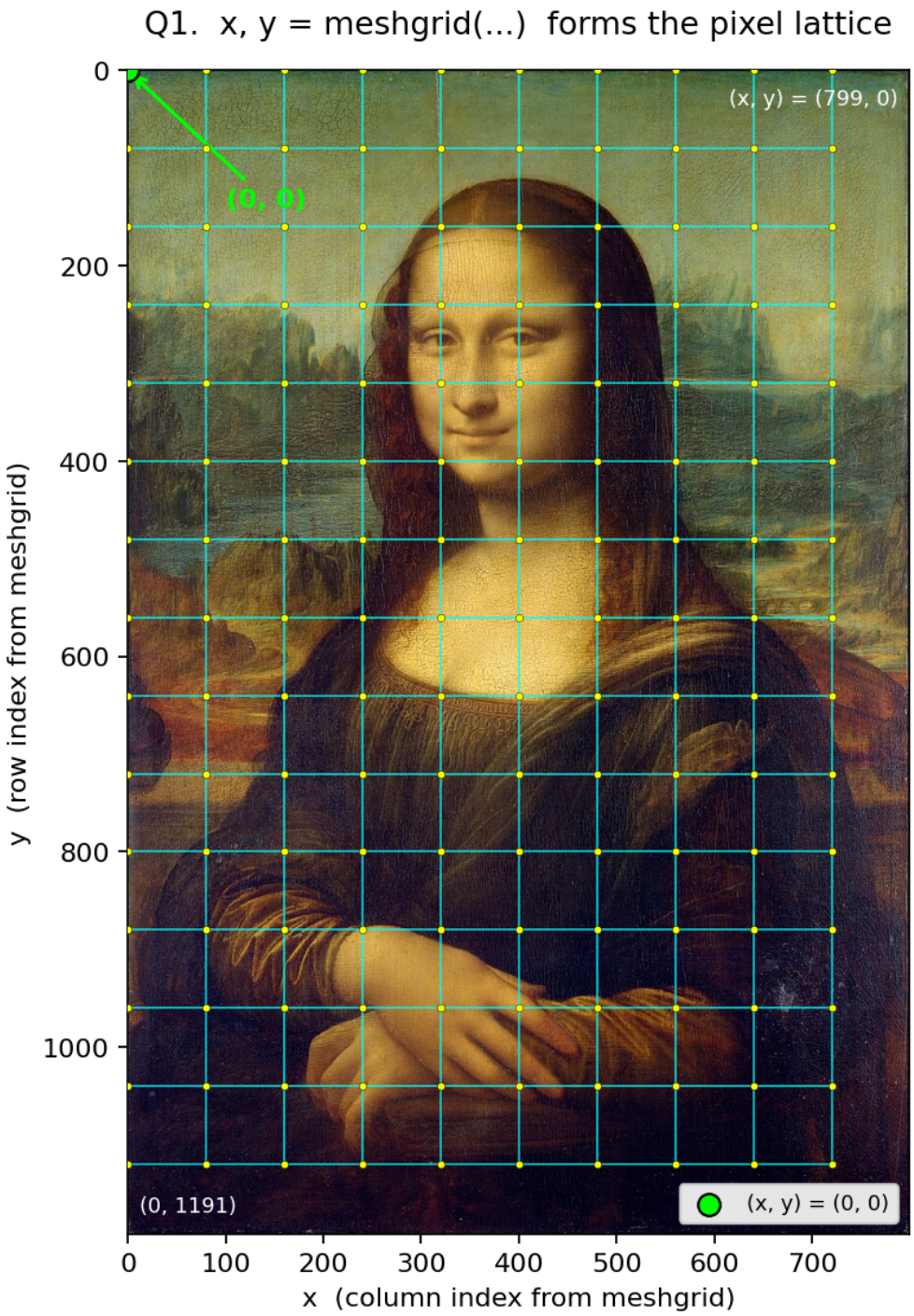


Figure 1: Pixel coordinates generated using `meshgrid` and their corresponding homogeneous coordinates.

3 Q2. Types and Order of Transformations

The transformation matrices in the provided code have the general form

$$H = \begin{bmatrix} \cos\theta & -\sin\theta & t_x \\ \sin\theta & \cos\theta & t_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{9}$$

The last row is $[0, 0, 1]$, so these are affine transformations. More specifically, depending on the values of θ, t_x, and t_y, each matrix used in this assignment represents either a pure translation or a pure rotation.

3.1 CASE 1

The first transformation in CASE 1 uses

$$\theta_1 = 0, \qquad t_x = 100, \qquad t_y = 100. \tag{10}$$

Therefore,

$$H_{0\to1} = \begin{bmatrix} 1 & 0 & 100 \\ 0 & 1 & 100 \\ 0 & 0 & 1 \end{bmatrix}, \tag{11}$$

which is a pure translation by $(100, 100)$ pixels.
The second transformation uses

$$\theta_2 = 10^\circ, \qquad t_x = 0, \qquad t_y = 0. \tag{12}$$

Thus,

$$H_{1\to2} = \begin{bmatrix} \cos 10^\circ & -\sin 10^\circ & 0 \\ \sin 10^\circ & \cos 10^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{13}$$

which is a pure rotation by 10° around the origin.
Therefore, CASE 1 performs

$$\boxed{\text{Translation} \to \text{Rotation}}. \tag{14}$$

The corresponding composite matrix is

$$H_{\text{CASE1}} = H_{1\to2} H_{0\to1}. \tag{15}$$

Let $\theta = 10^\circ$. Then,

$$H_{\text{CASE1}} = \begin{bmatrix} \cos\theta & -\sin\theta & 100(\cos\theta - \sin\theta) \\ \sin\theta & \cos\theta & 100(\sin\theta + \cos\theta) \\ 0 & 0 & 1 \end{bmatrix}. \tag{16}$$

For $\theta = 10^\circ$, the translation component is approximately

$$(81.12,\ 115.85). \tag{17}$$

3.2 CASE 2

In CASE 2, the first transformation uses

$$\theta_1' = 10^\circ, \qquad t_x' = 0, \qquad t_y' = 0. \tag{18}$$

Therefore,

$$H_{0\to1}' = \begin{bmatrix} \cos 10^\circ & -\sin 10^\circ & 0 \\ \sin 10^\circ & \cos 10^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}, \tag{19}$$

which is a pure rotation.
The second transformation is

$$\theta_2' = 0, \qquad t_x' = 100, \qquad t_y' = 100, \tag{20}$$

so

$$H_{1\to 2}' = \begin{bmatrix} 1 & 0 & 100 \\ 0 & 1 & 100 \\ 0 & 0 & 1 \end{bmatrix}, \tag{21}$$

which is a pure translation.
Therefore, CASE 2 performs

$$\boxed{\text{Rotation} \to \text{Translation}}. \tag{22}$$

Its composite matrix is

$$H_{\text{CASE2}} = H_{1\to 2}' H_{0\to 1}', \tag{23}$$

which gives

$$H_{\text{CASE2}} = \begin{bmatrix} \cos\theta & -\sin\theta & 100 \\ \sin\theta & \cos\theta & 100 \\ 0 & 0 & 1 \end{bmatrix}. \tag{24}$$

3.3 Comparison of CASE 1 and CASE 2

Both CASE 1 and CASE 2 consist of a rotation and a translation, so their overall transformations are Euclidean transformations. However, the results are different because the order of the transformations is different.

In general,

$$RT \neq TR. \tag{25}$$

For CASE 1, the translation is performed first and is subsequently affected by the rotation. For CASE 2, the rotation is performed first, and the translation is then applied directly.

This can also be seen from the two composite matrices. In CASE 1, the translation part becomes approximately $(81.12, 115.85)$, whereas in CASE 2 it remains $(100, 100)$.

Therefore, the order of transformations is important.

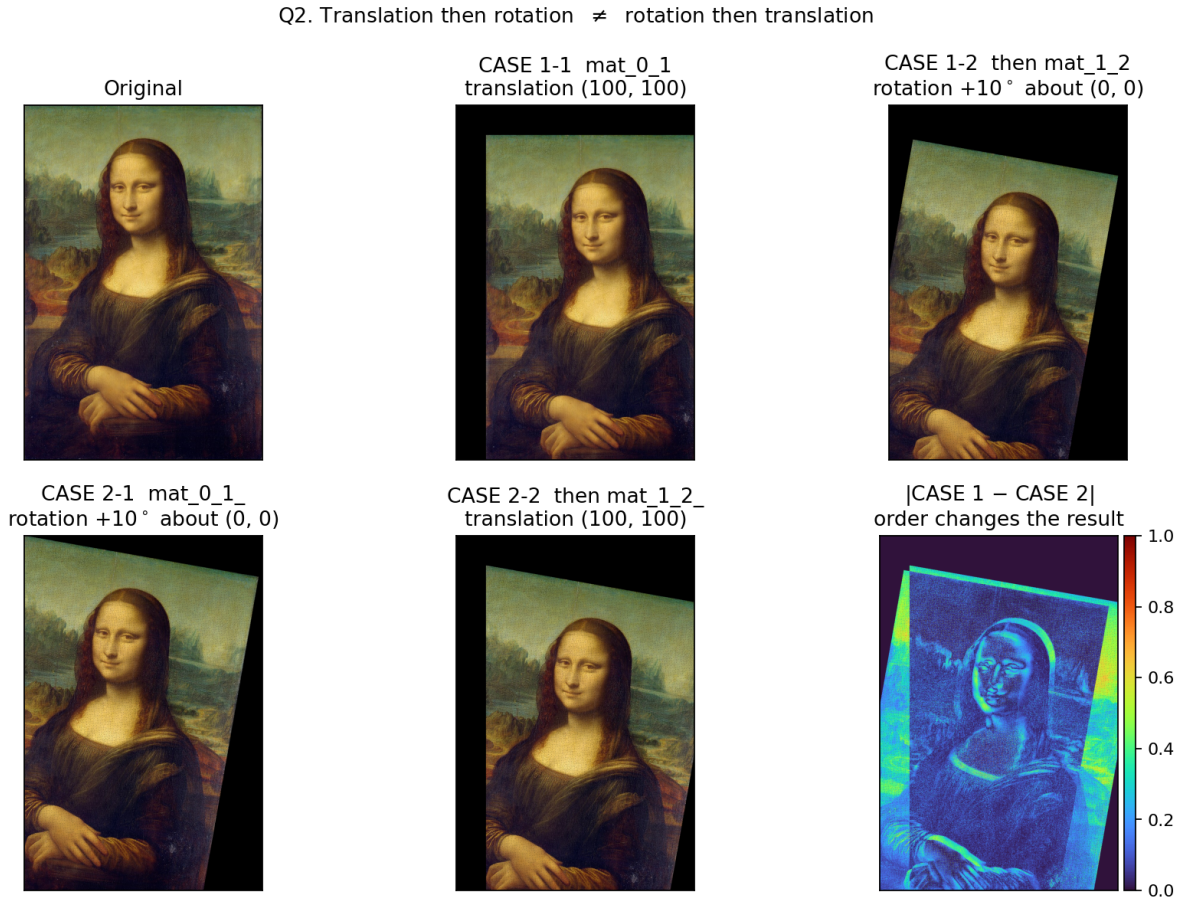


Figure 2: Comparison of CASE 1 and CASE 2. The two cases apply the same rotation and translation in different orders.

When the transformations are applied sequentially to rasterized images, the intermediate image is converted to integer pixel locations at each step. Therefore, a one-shot application of the composite matrix represents the same geometric coordinate transformation, but the resulting rasterized image can differ slightly because of rounding and repeated rasterization.



Figure 3: Comparison related to the composite transformation. Small pixel-level differences can occur because sequential transformations rasterize the image at each step.

4 Q3. Rotation Center

The rotation matrix used in the code is

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{26}$$

For a point

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

the transformed point is

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = R(\theta) \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}. \tag{27}$$

Since there is no translation component, the point $(0, 0)$ remains fixed. Therefore, the rotation is performed around the origin.

For an image coordinate system, the origin corresponds to the top-left corner of the image. Thus, the rotation in the provided code is around the top-left corner rather than around the center of the image.

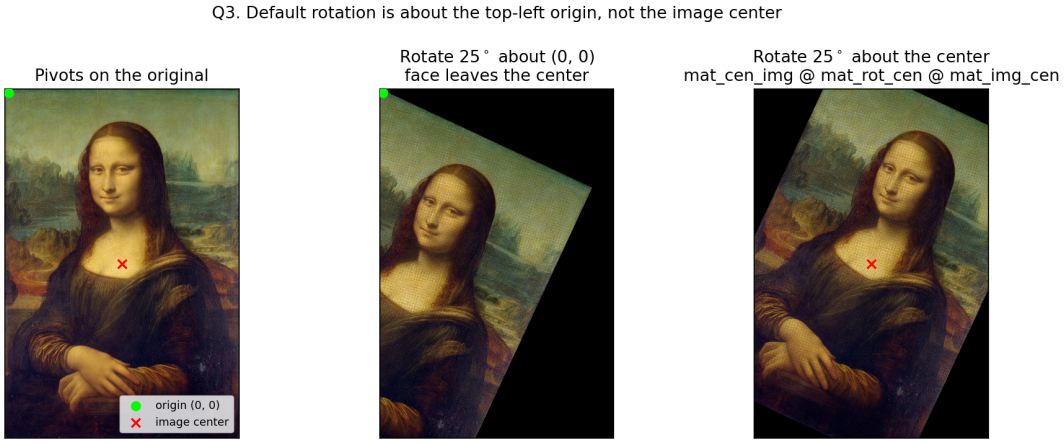


Figure 4: Rotation around the origin and the image center.

4.1 Rotation Around the Image Center

To rotate an image around its center, the point must first be translated to the origin, then rotated, and finally translated back.

For an image with width W and height H, using pixel coordinates from 0 to $W-1$ and 0 to $H-1$, the geometric center is

$$c_x = \frac{W-1}{2}, \qquad c_y = \frac{H-1}{2}. \tag{28}$$

For the input image,

$$W = 800, \qquad H = 1192, \tag{29}$$

so

$$(c_x, c_y) = (399.5, 595.5). \tag{30}$$

The transformation for rotation around the image center is

$$H_{\text{center}} = T(c_x, c_y)R(\theta)T(-c_x, -c_y). \tag{31}$$

Expanding the matrices gives

$$H_{\text{center}} = \begin{bmatrix} \cos\theta & -\sin\theta & c_x(1-\cos\theta) + c_y\sin\theta \\ \sin\theta & \cos\theta & -c_x\sin\theta + c_y(1-\cos\theta) \\ 0 & 0 & 1 \end{bmatrix}. \tag{32}$$

This shows that changing the rotation center can be achieved using translations before and after the rotation.

The exact geometric center is $(399.5, 595.5)$. The nearest integer pixel location is $(400, 596)$.

5 Q4. Projective Transformation

A general projective transformation is represented using a 3×3 homography matrix H:

$$\tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1. \tag{33}$$

The source point is represented in homogeneous coordinates as

$$\tilde{\mathbf{p}}_1 = \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}. \tag{34}$$

After applying the transformation,

$$\tilde{\mathbf{p}}_2 = \begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = H \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}. \tag{35}$$

The Cartesian coordinates are then obtained by dividing by the third homogeneous coordinate:

$$x_2 = \frac{\tilde{x}_2}{\tilde{z}_2}, \qquad y_2 = \frac{\tilde{y}_2}{\tilde{z}_2}. \tag{36}$$

This perspective division is the main additional step compared with the affine transformation used in `apply_transformation`.

5.1 Implementation

The corresponding implementation is shown below. It follows the same forward-mapping structure as `apply_transformation`, with the additional perspective divide

$$x_2 = \frac{\tilde{x}_2}{\tilde{z}_2}, \qquad y_2 = \frac{\tilde{y}_2}{\tilde{z}_2}.$$

```
def apply_projective_transformation(image, transformation_matrix):
    rows, cols, channels = image.shape
    if transformation_matrix.shape != (3, 3):
        raise ValueError("Transformation␣matrix␣must␣be␣3x3␣in␣size.")

    transformed_image = np.zeros_like(image)
```

```
    # p2 = H p1
    x1, y1 = np.meshgrid(np.arange(cols), np.arange(rows))
    p1 = np.stack(
        [x1.ravel(), y1.ravel(), np.ones(rows * cols, dtype=np.float64)],
        axis=0,
    )

    H = transformation_matrix.astype(np.float64)
    p2 = H @ p1
    z2 = p2[2]
    valid_z = np.abs(z2) > 1e-8

    new_x = np.zeros(z2.shape[0], dtype=int)
    new_y = np.zeros(z2.shape[0], dtype=int)
    new_x[valid_z] = (p2[0, valid_z] / z2[valid_z]).astype(int)
    new_y[valid_z] = (p2[1, valid_z] / z2[valid_z]).astype(int)

    valid = (
        valid_z
        & (new_x >= 0)
        & (new_x < cols)
        & (new_y >= 0)
        & (new_y < rows)
    )

    transformed_image[new_y[valid], new_x[valid]] = \
        image.reshape(-1, channels)[valid]

    return transformed_image
```

The projective transformation was implemented using the same forward-mapping idea as the provided `apply_transformation` function.

First, the coordinates of the source image pixels are generated using `meshgrid` and represented as homogeneous coordinates. The transformation matrix is then directly applied:

$$\tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1. \tag{37}$$

The third coordinate is used for perspective division. The resulting coordinates are rounded to integer pixel locations, and only coordinates inside the image boundary are copied to the output image.

In contrast to an inverse-mapping implementation, this method does not calculate H^{-1}. The source pixels are directly mapped to their new destination locations.

Because this is a forward-mapping procedure, several source pixels can map to the same destination pixel, while some destination pixels may not receive any source pixel. As a result, small holes or empty pixels can appear in the transformed image. These holes are caused by the rasterization process.

5.2 Parallel Lines

To demonstrate the property of projective transformations, two initially parallel horizontal lines were transformed using a projective transformation.

Before the transformation, both lines have zero slope:

$$m_{\text{upper}} = 0, \qquad m_{\text{lower}} = 0. \tag{38}$$

The endpoints of each line were then transformed using the homography matrix H. The slope after transformation was calculated from the transformed endpoints.

The measured slopes are shown in Table 1.

Table 1: Slopes of the two parallel lines before and after the projective transformation.

Line	Before	After
Upper (red)	0.000	$\mathbf{-0.222}$
Lower (blue)	0.000	$\mathbf{+0.013}$

The two lines are parallel before the transformation because they have the same slope. After the transformation, their slopes are different:

$$-0.222 \neq +0.013. \tag{39}$$

Therefore, the two lines are no longer parallel.

This demonstrates an important difference between affine and general projective transformations. Affine transformations preserve parallelism, whereas a general projective transformation does not.

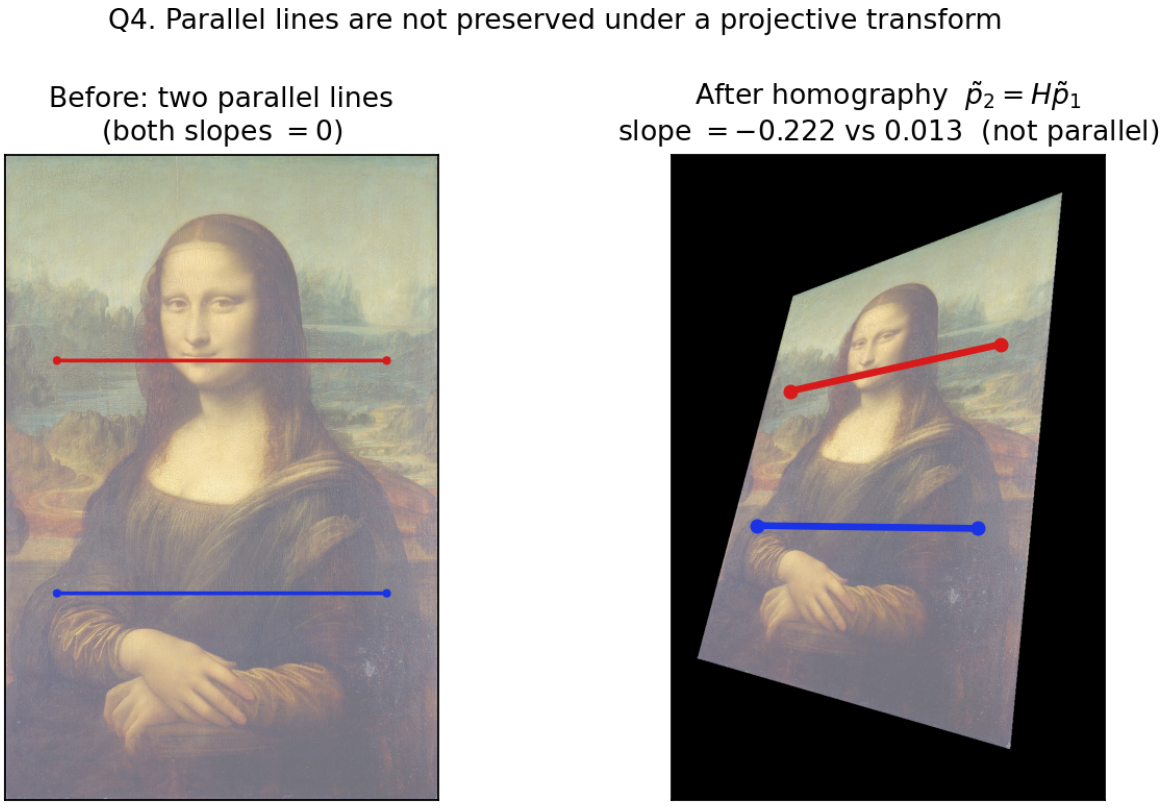


Figure 5: Two initially parallel lines before and after the projective transformation. The transformed lines have different slopes and are therefore no longer parallel.

5.3 Projective Effect on a Grid

A grid was also transformed to visualize the geometric effect of the projective transformation.

Under a projective transformation, straight lines remain straight. However, parallel lines do not necessarily remain parallel. Therefore, the transformed grid can show a perspective-like effect in which grid lines appear to converge.

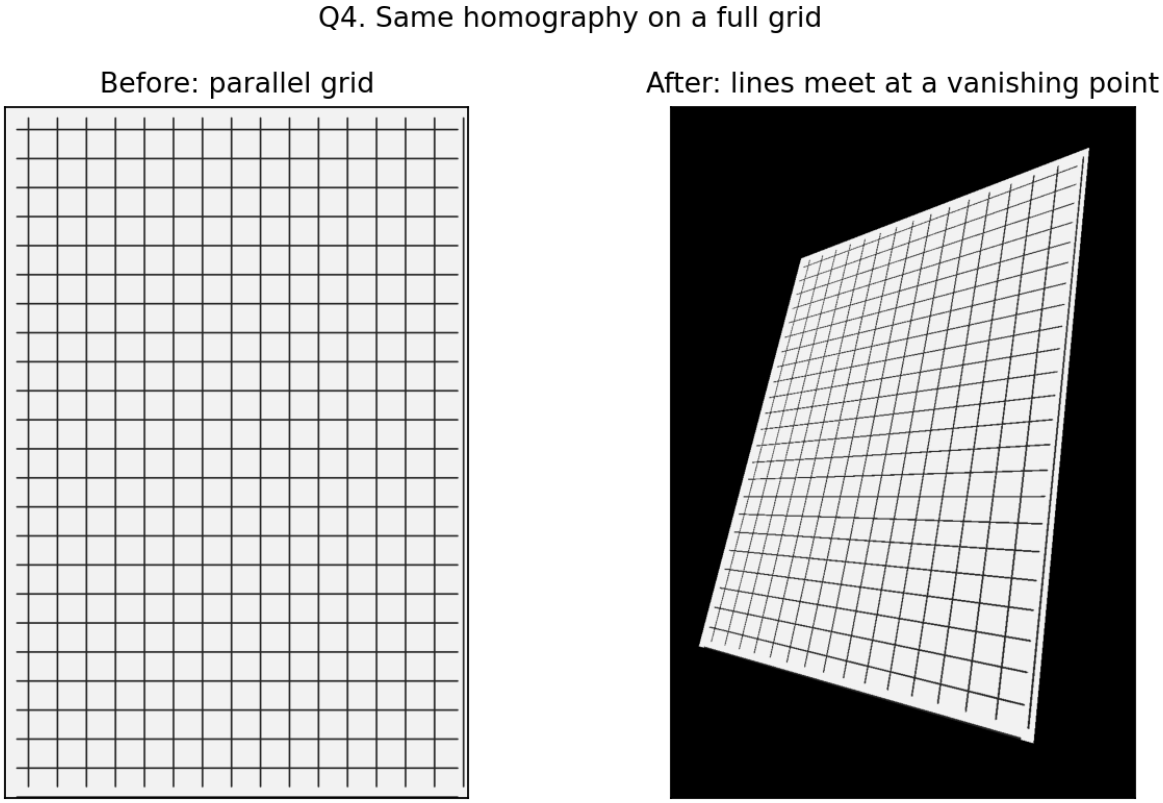


Figure 6: Projective transformation of a grid. Straight grid lines remain straight, while their parallelism is not necessarily preserved.

6 Summary

This assignment demonstrated 2D transformations using homogeneous coordinates.

First, `meshgrid` was used to generate the coordinate of every image pixel efficiently. These coordinates were converted into homogeneous coordinates so that they could be transformed using 3×3 matrices.

Second, the two transformation cases showed that the order of rotation and translation is important. CASE 1 applies translation followed by rotation, while CASE 2 applies rotation followed by translation. Since matrix multiplication is not commutative, the two cases produce different results.

Third, the rotation matrix in the provided code rotates around the origin, which corresponds to the top-left corner of the image. To rotate around the image center, translations before and after the rotation are required.

Finally, a projective transformation was implemented using forward mapping:

$$\tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1, \qquad x_2 = \frac{\tilde{x}_2}{\tilde{z}_2}, \qquad y_2 = \frac{\tilde{y}_2}{\tilde{z}_2}. \tag{40}$$

The experiment with two parallel lines showed that their slopes changed from $(0, 0)$ to approximately $(-0.222, +0.013)$. Hence, the lines were no longer parallel after the projective

transformation. This confirms experimentally that general projective transformations do not preserve parallelism.